

DIGITAL COGNITIVE ARCHITECTURE

A Complete Step-by-Step Builder's Guide

From First Principles to Autonomous Intelligence

What to Build · Why to Build It · How to Build It · What to Use

Chapter 0 — What Is a Cognitive Architecture?

Before writing a single line of code, you need a clear mental model of what you are actually building. This is not an AI wrapper. It is not a chatbot. It is not a monitoring dashboard. Those things are outputs — small, disposable applications. What you are building is the substrate that makes any of them possible.

The Core Idea

A cognitive architecture is a system that maintains a persistent, evolving, internally consistent model of an environment — and uses that model to observe, reason, plan, and act — continuously, over time, without being restarted or re-prompted.

Think of how a human expert operates. A senior engineer doesn't wake up every morning with zero memory. They carry a mental model of their system: which services are fragile, which team made what change last week, what happens to the database when traffic spikes on Fridays. When something breaks, they don't start from scratch — they update their existing model and reason from it. That accumulated, continuously updated understanding is what makes them effective. A cognitive architecture does exactly this, but for machines.

KEY
IN
SIGHT

The goal is not to build a smarter chatbot. The goal is to build a system that knows its environment the way an expert knows theirs — with history, context, relationships, and the ability to predict what happens next.

Why This Is Not a Microservices Health Checker

The document you have been working from uses API monitoring as the introductory example. That is only because it is the simplest environment to describe. The architecture itself is universal. The environment could be:

- A hospital — monitoring patient vitals, medication schedules, staff assignments, and equipment availability. The cognitive system understands that Patient A's deteriorating blood pressure correlates with the drug administered two hours ago, reasons about intervention options, and plans a care adjustment.
- A financial trading floor — ingesting market feeds, order books, news sentiment, macroeconomic signals, and position data. The cognitive system understands causal chains between events and executes risk-adjusted decisions faster than any human.
- A city's traffic network — perceiving vehicle density, signal states, weather, events, and historical patterns. The cognitive system builds a live world model of the entire city and plans signal timing to minimize aggregate congestion.
- A game world — where the cognitive architecture powers an NPC that genuinely learns the player's behavior, builds a model of the player's strategy, and adapts its own tactics accordingly.
- A scientific research pipeline — ingesting experimental results, literature, hypotheses, and resource availability. The cognitive system identifies patterns humans miss and proposes the next experiment to run.
- A logistics network — perceiving warehouse inventory, vehicle locations, weather, demand signals, and supplier states. The system plans routes and reorders inventory in response to a world model that updates in real time.
- A software development environment — perceiving code changes, test results, deployment states, developer activity, and incident history. The system understands the codebase as a living entity and reasons about risk, technical debt, and safe deployment windows.

In every case the same architecture applies. What changes is only the perception sources and the action vocabulary. The cognitive substrate — perception, memory, world model, reasoning, planning, execution, feedback — is identical.

The Seven Pillars of Cognitive Architecture

Every component you will build maps to one of these seven functions:

Pillar	What It Does
Perception	Continuously ingests raw signals from the environment and normalizes them into structured, machine-understandable events.
Memory	Stores and retrieves information across multiple timescales — from milliseconds (working) to years (semantic). Provides continuity.
World Model	Fuses memory and perception into a coherent, queryable representation of reality. Answers: what exists, what changed, why, and what might happen next.
Reasoning	Uses the world model to analyze situations, evaluate options, and reach conclusions. The LLM lives here — as one component, not the entire system.
Planning	Converts reasoning outputs into executable strategies. Decomposes goals into ordered sequences of actions with contingency handling.
Execution	Carries out actions in the environment — API calls, file writes, process spawning, agent delegation, physical actuation.
Feedback	Observes the consequences of actions, updates the world model, and closes the loop. This is what transforms a pipeline into an adaptive system.

How the Pillars Connect

These seven pillars are not independent modules. Each one feeds the next, and feedback connects the last back to the first. Here is the complete flow:

The environment emits raw signals — log entries, sensor readings, API responses, user actions, file changes. The Perception Layer ingests these continuously, normalizes them into a common structured format (the unified event schema), and routes them to the Memory system. Memory stores these events across four layers — working memory for immediate context, episodic memory for historical experience, semantic memory for relational knowledge, and procedural memory for learned strategies. The World Model continuously reads from all four memory layers and from the live perception stream to maintain a real-time, causally aware model of the environment. When a decision is needed, the Reasoning Engine queries the World Model for a compressed situation summary and uses an LLM (or a faster policy) to evaluate options and select an action. The Planning system converts that decision into a structured execution plan — a sequence of concrete steps with dependencies and fallbacks. The Execution Layer

carries out each step — making API calls, writing files, spawning sub-agents, sending messages. And finally, every executed action produces a new signal in the environment, which the Perception Layer picks up, which updates Memory, which updates the World Model — closing the loop and making the system adaptive.

WHY THE LOOP MATTERS

A system without the feedback loop is just an automation pipeline. It acts but never learns. The feedback loop is what transforms execution into intelligence — the system observes the consequences of its own decisions and updates its model of the world accordingly.

Chapter 1 — Phase 0: Build the Mental Model First

This is the most important phase. Most failed projects fail here — not because the builder couldn't code, but because they started coding before they understood what they were building. Spend real time here. It will save you weeks of rework.

What to Study and Why

1A — Cognitive Architectures (1 week)

Study: SOAR, ACT-R, Global Workspace Theory, Active Inference, Predictive Processing.

Why: These are decades of research into how cognition is organized. SOAR will show you how a system can have goals, sub-goals, and procedural memory. ACT-R will show you how different memory systems interact. Global Workspace Theory will show you how a central broadcast mechanism (which your world model approximates) enables coordinated intelligence. You are not implementing these directly — you are extracting their structural insights.

What to take away: The idea that cognition is not one thing but a structured set of cooperating systems, each with a distinct memory type and access pattern.

1B — World Models (1 week)

Study: Latent state modeling, predictive world models, scene graphs, environment simulation, JEPA (Joint Embedding Predictive Architecture by Yann LeCun).

Why: A world model is not a database. It is a dynamic, compressed representation of reality that supports prediction. Understanding this distinction is critical. Your world model needs to answer not just 'what is true now' but 'what is likely true in five minutes if no action is taken.' This is the difference between a dashboard and a cognitive system.

What to take away: The concept of latent state — the idea that you represent the world not as raw data but as a compressed structured model that captures the essential variables and their causal relationships.

1C — Agentic and Multi-Agent Systems (1 week)

Study: LangGraph, AutoGen, CrewAI, planner-executor architectures, distributed agent coordination.

Why: You need to understand how agents orchestrate execution — how a high-level goal gets decomposed into tasks, how tasks get assigned to specialized executors, and how the results are aggregated. LangGraph is worth particular attention because it models agent behavior as a stateful graph, which aligns well with the architecture you are building.

What to take away: The distinction between a planner agent (which decides what to do) and an executor agent (which does it). This separation is essential for building systems that can reason about plans before committing to them.

1D — Graph Intelligence (1 week)

Study: Graph databases (Neo4j), graph neural networks, knowledge graphs, relational reasoning, RDF/ontologies.

Why: The world is not a set of independent rows in a table. Things have relationships. Services depend on other services. Users belong to organizations. Events cause other events. A graph is the natural data structure for representing a world model because it captures these relationships explicitly. You cannot do causal reasoning over a relational database the way you can over a knowledge graph.

What to take away: How to model entities, relationships, and properties in a graph. How to query for paths, neighbors, and subgraphs. How Neo4j's Cypher query language works.

1E — Temporal Intelligence (1 week)

Study: Event streams (Kafka), temporal databases, sequence modeling, time-series analysis, event sourcing pattern.

Why: Everything in a live environment happens in time. A fact that is true now may not be true in ten minutes. A system that treats the world as a static snapshot will always be reasoning about a stale model. Temporal intelligence means understanding not just the current state but the trajectory — how state has been evolving and where it is heading.

What to take away: The event sourcing pattern — the idea that the authoritative record is a stream of events, not a snapshot of current state. Current state is always derived from the event history.

Don't try to master all of this before building. Study each area enough to understand its core concepts and vocabulary, then start building. Return to deepen your understanding as you encounter specific problems.

Chapter 2 — Phase 1: Define the Unified Event Schema

This is the first thing you build, and it is the most important engineering decision in the entire project. Every other component depends on it. Get this wrong and you will spend months retrofitting.

What Is the Unified Event Schema?

The unified event schema is the common language that every component in your architecture speaks. Raw signals from the environment are infinitely varied — a log line looks nothing like a CPU metric, which looks nothing like a user click event, which looks nothing like a database query result. The schema is the normalization contract that transforms all of them into a single, consistent, machine-understandable format.

Why This Must Come First

Consider what happens without it. Your perception layer produces one format. Your memory system expects another. Your world model queries a third. Every time you add a new signal source, you write a new adapter. Every time you change the schema, you break everything downstream. By defining a single schema first, you write the adapter once per source and every downstream component works automatically.

The Schema Structure

Every event in your system — regardless of source — must conform to this envelope:

Field	Purpose and Values
event_id	UUID. Unique identifier for this specific event. Used for deduplication and tracing.
timestamp	ISO 8601 with timezone. When the event actually occurred in the environment (not when it was processed).

ingested_at	ISO 8601. When the perception layer received and processed this event. Lag between timestamp and ingested_at indicates processing delay.
source_type	Enum: log metric api queue file user_event database sensor agent_event. Tells downstream systems what kind of signal this is.
source_id	String. Identifies the specific source — e.g., 'auth-service', 'db-primary', 'user-dashboard'. Used to build the entity graph.
event_type	String. Semantic label for what happened — e.g., 'database_timeout', 'cpu_spike', 'order_placed', 'patient_vitals_alert'. This is the normalized vocabulary.
severity	Enum: critical high medium low info. Used by the memory router and the situation assessor to prioritize.
payload	JSON object. Source-specific structured data. Content varies by event_type but is always structured, never raw string.
entity_refs	Array of entity IDs from the knowledge graph that this event relates to. Links events to graph nodes.
confidence	Float 0.0–1.0. How certain is the perception layer about this event's interpretation. Sensor noise, ambiguous logs, or inferred events have lower confidence.
tags	Array of strings. Flexible labels for filtering and routing — e.g., ['production', 'database', 'latency'].

A Concrete Example

Here is a raw log line and its transformation into the schema:

RAW INPUT	2024-01-15T14:23:11Z [ERROR] auth-service: connection to postgres-primary timed out after 5000ms, pool exhausted (pool_size=20, waiting=47)
------------------	---

```
{ "event_id": "evt_a1b2c3", "timestamp": "2024-01-15T14:23:11Z", "source_type": "log",
  "source_id": "auth-service", "event_type": "database_connection_timeout", "severity": "high",
  "payload": { "database": "postgres-primary", "timeout_ms": 5000, "pool_size": 20,
    "waiting_connections": 47 }, "entity_refs": ["svc:auth-service", "db:postgres-primary"],
  "confidence": 0.97, "tags": ["production", "database", "connection-pool"] }
```

Notice what happened. The raw text became a structured object with explicit entity references and a semantic event type. Any component downstream can now query: 'show me all events referencing entity db:postgres-primary in the last 10 minutes' — without parsing log strings.

How to Build It

1. Define your event_type vocabulary. Spend time on this. Create a spreadsheet listing every type of event your environment can produce and give each one a semantic name. Group them by category.
2. Define your entity namespace. Decide on a consistent scheme for entity IDs — e.g., 'svc:service-name', 'db:database-name', 'usr:user-id'. Every entity that appears in events must have an ID in this namespace.
3. Write a schema validator. Use JSON Schema or Pydantic. Every event that cannot be validated against the schema is logged as a perception failure and sent back for re-processing.
4. Build one normalizer per source type. Each normalizer is a small function that takes raw input from one source and outputs a validated event. Start with your two or three most important sources.

Tools to Use

Tool	Purpose
Pydantic (Python)	Define and validate the event schema as a typed Python class. Gives you automatic validation, serialization, and documentation.
Apache Avro or Protobuf	For high-throughput environments, use a binary schema format instead of JSON. Avro integrates natively with Kafka.

JSON Schema

Lightweight schema definition if you prefer language-agnostic validation.

Chapter 3 — Phase 2: Build the Perception Layer

The perception layer is the sensory system of your cognitive architecture. It is always on, always listening, and its sole responsibility is to convert raw environmental signals into normalized events that the rest of the system can understand.

What the Perception Layer Is

The perception layer is not a batch job. It is not a cron task. It is a continuous, streaming, event-driven process that:

- Connects to every signal source in your environment
- Receives or polls for new data in real time or near-real time
- Runs each raw input through the appropriate normalizer
- Validates the output against the unified event schema
- Assigns entity references, severity, and confidence scores
- Publishes validated events to the central event bus
- Tracks its own health and reports perception gaps (missed signals, processing lag, validation failures)

The Central Event Bus

The event bus is the circulatory system of the entire architecture. Every normalized event flows through it. Every downstream component — memory, world model, reasoning — subscribes to it. This decoupling is critical: the perception layer does not need to know who is consuming its events, and consumers do not need to know how events were produced.

Use Apache Kafka or Redpanda as your event bus. Both provide durable, ordered, replayable event streams with consumer group support. The ability to replay events from a specific point in time is invaluable during development and debugging — you can replay the last hour of events through a new version of your world model to see how it would have performed.

Signal Sources and Their Adapters

Source Type	What to Ingest	Adapter Approach
-------------	----------------	------------------

Application Logs	Error messages, warnings, request/response logs, exception traces	Log agent (Filebeat/Fluent Bit) streams to Kafka; normalizer parses log format and extracts event_type
System Metrics	CPU, memory, disk I/O, network throughput, process counts	Prometheus scrapes metrics; a bridge publishes metric threshold crossings as events
APIs and Services	HTTP response codes, latency, payload content, health check results	Polling adapter with configurable interval; change-detection logic to emit events only on state change
Database State	Query latency, connection pool usage, replication lag, table sizes	Database exporter streams to Kafka; normalizer maps metrics to semantic events
Message Queues	Queue depth, consumer lag, message age, dead-letter queue size	Queue monitor subscribes to admin API; emits events when thresholds crossed
File System	File creation, modification, deletion, permission changes	inotify (Linux) or fsevents (Mac) provides real-time file system events
User Events	Clicks, form submissions, navigation, session start/end	Frontend SDK posts to perception API endpoint; normalizer extracts intent and context
Sensor Data	Temperature, pressure, GPS coordinates, accelerometer, camera frames	Hardware SDK or MQTT broker feeds raw sensor data to normalizer
External APIs	Weather, market data, news feeds, third-party service status	Polling adapter with change detection; rate-limit-aware retry logic
Agent Events	Actions taken by sub-agents, reasoning outputs, plan completions	Agent SDK publishes structured agent_event type directly to event bus

The Normalizer Pattern

Each adapter follows the same pattern. It is a pure function: raw input in, validated event out. If it cannot produce a valid event, it emits a `perception_failure` event instead. It never silently drops data.

The normalizer's most important job is extracting the `event_type` and `entity_refs`. These two fields are what allow the rest of the system to reason relationally. A log line that says 'connection failed' is noise. An event with `event_type: database_connection_timeout` and `entity_refs: [svc:auth-service, db:postgres-primary]` is knowledge.

The Confidence Score

Not all signals are equally reliable. A metric threshold crossing from Prometheus is highly reliable — confidence 0.98. An event_type inferred from an unstructured log line using regex matching is less reliable — confidence 0.75. An event reconstructed from indirect evidence (e.g., inferring a service crash from the sudden absence of heartbeat signals) is even less reliable — confidence 0.55. Every downstream component must respect these confidence scores. The world model should weight low-confidence events accordingly. The reasoning engine should flag low-confidence situations and ask for more evidence before acting.

What to Build First

Start with the two or three signal sources that are most important to your target environment. Build a complete pipeline: adapter, normalizer, validator, event bus publisher. Run it for a week and inspect the output. You will discover that your schema needs refinement, your event_type vocabulary is incomplete, and your entity namespace has gaps. Fix these before adding more sources.

Tech Stack for the Perception Layer

Component	Recommended Tools
Event bus	Apache Kafka (production) or Redpanda (simpler setup, Kafka-compatible API)
Log ingestion	Fluent Bit (lightweight agent) or Filebeat; streams to Kafka topic
Metrics bridge	Prometheus + custom exporter that converts metric alerts to schema events
API / service polling	FastAPI background tasks or Celery beat for scheduled polling
Schema validation	Pydantic v2 for Python; fast, typed, auto-documenting
Event bus client	confluent-kafka-python or aiokafka for async Python
Perception API	FastAPI endpoint for sources that push (user events, agent events, webhooks)

Chapter 4 — Phase 3: Build the Memory System

Memory is what gives your cognitive architecture continuity. Without it, every reasoning step starts from zero. With it, the system accumulates understanding — it knows what has happened before, what patterns exist, how things relate, and what strategies have worked.

Why Four Memory Types?

Different questions require different types of storage. 'What is happening right now?' is a different question from 'What happened last Tuesday?' which is different from 'What is the relationship between service A and database B?' which is different from 'How do we handle a connection pool exhaustion?'. Each requires a different storage mechanism optimized for its access pattern. Trying to use a single database for all four creates a system that is mediocre at everything.

Memory Layer 1: Working Memory

Working memory holds the current active context. It is the scratchpad of the system — fast, volatile, and limited in size. Its purpose is to make the current situation immediately available to the reasoning engine without requiring a database query.

- What it stores: current task, active goals, recent events (last 5–10 minutes), ongoing plan state, current world model snapshot for active reasoning
- Access pattern: reads and writes happen in milliseconds; everything expires after a short TTL
- Technology: Redis. Use Redis hashes for structured objects, Redis sorted sets for time-ordered event windows, Redis pub/sub for internal component communication
- TTL strategy: working memory events expire after 15 minutes by default. Active goals and plans persist until explicitly completed or cancelled

Memory Layer 2: Episodic Memory

Episodic memory is the system's autobiographical record. Every event that flows through the perception layer is stored here, with full context, forever. It is the source of truth for what happened and when. It is also the training data for future predictive models.

- What it stores: every validated event from the perception layer; agent actions and their outcomes; reasoning traces and decisions; plan executions and their results
- Access pattern: append-only writes; complex temporal queries (events between T1 and T2, events matching pattern X in the last 30 days, similar events to this one)
- Technology: PostgreSQL with TimescaleDB extension for time-series queries, plus a vector database (Qdrant or Pinecone) for semantic similarity search
- Why vector search: when the reasoning engine needs to find 'events similar to this one' to retrieve relevant past experience, semantic similarity over embeddings is far more powerful than text search or SQL pattern matching

Memory Layer 3: Semantic Memory

Semantic memory is the system's knowledge of how things relate to each other. It is not about what happened (episodic) or what is happening now (working) — it is about the persistent structural reality of the environment. Which services depend on which databases. Which users have access to which systems. Which failure modes are known to correlate.

- What it stores: entities (services, users, databases, devices, agents) and their properties; relationships between entities (depends_on, owns, triggers, communicates_with); known causal patterns (if X then typically Y); domain knowledge accumulated over time
- Access pattern: graph traversal queries (what does service A depend on, transitively?); pattern matching (find all services that share a dependency on this database); writes happen when new entities are discovered or relationships change
- Technology: Neo4j for the knowledge graph; Cypher query language for expressive graph queries
- How it gets populated: the perception layer's entity_refs field drives graph updates. Every event that mentions an entity either creates that entity node or updates its last_seen timestamp. Every event that connects two entities creates or reinforces the relationship edge between them

Memory Layer 4: Procedural Memory

Procedural memory stores how to do things. It is the library of strategies, playbooks, and execution templates that the planning system draws from. When the planning system needs to handle a database connection pool exhaustion, it looks here for the established procedure rather than reasoning from scratch every time.

- What it stores: execution playbooks (step-by-step procedures for known situations); planning templates (parameterizable action sequences); learned strategies (which approaches have succeeded and failed in which contexts)

- Access pattern: retrieval by situation type or goal type; updates when new strategies are learned from feedback; version history to track how procedures evolve
- Technology: PostgreSQL for structured storage; vector search for retrieving procedures by semantic similarity to the current situation

The Memory Router

The memory router is a small but critical component that sits between the event bus and the memory layers. Every event that arrives from the perception layer passes through the router, which decides which memory layers should receive it.

The routing logic is deterministic and fast. It does not use an LLM. It applies a set of rules: all events go to episodic memory; events with entity_refs update the semantic graph; events above a severity threshold go to working memory; events that match known playbook triggers update procedural memory's activation state.

Tech Stack for Memory

Memory Type	Technology
Working Memory	Redis 7+ — use hashes, sorted sets, and TTL-based expiration. Aim for sub-10ms read latency.
Episodic Memory (time-series)	PostgreSQL 16 + TimescaleDB — hypertables give automatic partitioning by time, fast range queries.
Episodic Memory (semantic search)	Qdrant — open source, fast, runs locally. Use text-embedding-3-small to embed event descriptions.
Semantic Memory (graph)	Neo4j Community Edition — free, full Cypher support, browser-based visualization for debugging.
Procedural Memory	PostgreSQL with JSONB columns for playbook steps; Qdrant for similarity-based retrieval.

Chapter 5 — Phase 4: Build the Knowledge Graph

The knowledge graph is the most intellectually important component in the architecture. It is what allows the system to reason relationally — to understand not just what happened, but what the consequences might be, because it knows how everything connects to everything else.

What a Knowledge Graph Is

A knowledge graph is a network of nodes (entities) and edges (relationships), where both nodes and edges can carry properties. It is the digital equivalent of the expert engineer’s mental map: they know that auth-service talks to postgres-primary, which is replicated to postgres-replica, which is also read by the analytics pipeline, which runs on the same cluster as the ML inference service.

When auth-service shows high latency, that expert does not just look at auth-service in isolation. They mentally traverse the graph: is the database the cause? If the database is slow, does that affect anything else? Yes — the analytics pipeline, and through it the ML service. This relational traversal is exactly what the knowledge graph enables computationally.

The Graph Schema

Design your node types and relationship types before writing any code. Here is a starting schema that works across most environments:

Node Types

Node Type	What It Represents
Service	A software service or application. Properties: name, version, health_status, tier, owner_team, last_seen.
Database	A data store of any kind. Properties: type, host, size_gb, replication_factor, backup_status.
User	A human or system user. Properties: id, role, last_active, access_level.

Agent	A cognitive sub-agent in the system. Properties: id, type, status, current_task.
Task	A unit of work being tracked. Properties: id, goal, status, priority, created_at.
Event	A significant past event (linked from episodic memory for causal chains). Properties: event_id, event_type, timestamp.
Resource	A compute or infrastructure resource. Properties: type, capacity, utilization, region.
Concept	Domain knowledge — a known failure mode, a business rule, a learned pattern. Properties: name, description, confidence.

Relationship Types

Relationship	Meaning
DEPENDS_ON	Directed. A depends on B to function. Critical for blast radius analysis — if B fails, all nodes with DEPENDS_ON B are affected.
COMMUNICATES_WITH	Bidirectional or directed. A sends requests to B. Used for tracing request paths.
OWNS	A is responsible for B. Used to route alerts and decisions to the right owner.
TRIGGERS	Event or service A causes event or action B. Causal chain representation.
PART_OF	A is a component of B. Hierarchical composition.
SIMILAR_TO	A and B have shown similar behavior patterns. Learned from episodic memory.
HISTORICALLY_CORRELATED	When A changes, B tends to change too. Learned from temporal analysis.
EXECUTES	Agent A executes task B.
RESOLVES	Event or action A resolved situation B. Used to learn what works.

How the Graph Stays Current

The graph is not manually maintained. It is continuously updated by the perception layer through the entity_refs field. Here is the update logic:

5. When an event arrives with entity_refs [svc:auth-service, db:postgres-primary]: ensure both nodes exist in the graph (create if not); update their last_seen timestamps; if the event_type implies a relationship (e.g., database_connection_timeout implies auth-service DEPENDS_ON postgres-primary), create or strengthen that relationship edge.
6. Relationship edges carry a weight property that increases with each confirming event and decays over time when not observed. A dependency that has not been seen in 30 days has a low weight and may be pruned.
7. New entity types are added dynamically. The graph does not have a fixed schema — it grows with the environment.

Why This Enables Causal Reasoning

Consider this query in Neo4j's Cypher language:

C
Y
P
H
E
R
Q
U
E
R
Y

```
MATCH (e:Event {event_type: 'database_connection_timeout'})-[:AFFECTS]->(s:Service)
WHERE e.timestamp > datetime() - duration('PT10M') RETURN s.name, count(e) as
impact_count ORDER BY impact_count DESC
```

This returns every service affected by database timeouts in the last 10 minutes, ranked by how many times they were affected. This is blast radius analysis — understanding the full scope of impact from a single failure — and it is trivial to express in a graph but extremely difficult in a relational database.

K
E
Y
P
R
I
N
C
I
P
L
E

The knowledge graph transforms individual events into systemic understanding. It is the difference between knowing 'the database is slow' and knowing 'the database is slow, and these 7 services depend on it, and 3 of them are customer-facing, and the on-call engineer for the database team is user:alice'.

Chapter 6 — Phase 5: Build Temporal Awareness

Temporal awareness is what gives your cognitive architecture a sense of time — the ability to understand not just what is true now, but how things have been changing, at what rate, and where they are heading. Without this, your system is perpetually surprised by the present. With it, your system anticipates the future.

The Three Temporal Dimensions

Your system needs to reason across three time horizons simultaneously:

Horizon	What It Requires
Recent past (seconds to minutes)	Working memory with time-ordered event windows. Used for immediate pattern detection and situational awareness.
Historical past (hours to months)	Episodic memory with time-series indexing. Used for trend analysis, anomaly detection against baseline, and identifying recurring patterns.
Anticipated future (minutes to hours)	World model with predictive state projection. Used for proactive planning before problems escalate.

What to Track

For each entity in your knowledge graph, maintain a temporal state profile:

- Current value — the most recently observed value of each tracked metric
- Rate of change — is this value increasing, decreasing, or stable? At what rate?
- Trend direction — is the rate of change itself accelerating?
- Deviation from baseline — how far is the current value from its typical range at this time of day/week?
- Time since last event — how long has it been since this entity was observed? Silence can be a signal.

Temporal Pattern Detection

Build a pattern detector that runs continuously over the event stream and identifies:

- Spike patterns — a metric crosses a threshold suddenly. High confidence, immediate action.
- Gradual drift patterns — a metric has been slowly moving outside its normal range for the past hour. Often missed by threshold alerts but caught by trend analysis.
- Correlation patterns — two metrics that are not normally correlated have started moving together. Strong signal of a shared underlying cause.
- Absence patterns — a normally frequent event type has not appeared for N minutes. Service may be down or disconnected.
- Recurrence patterns — a type of event that appeared last Tuesday is appearing again today at the same time. Possible scheduled process, possible recurring bug.

Connecting Temporal Awareness to the Graph

Every temporal pattern detected should update the knowledge graph. A detected correlation between two metrics creates or reinforces a **HISTORICALLY_CORRELATED** relationship. A recurrence pattern creates a **Concept** node (**known_recurring_event**) linked to the affected entities. This is how learned knowledge about time-based patterns gets encoded into the relational structure of the world model.

Tools for Temporal Intelligence

Tool	Purpose
TimescaleDB (PostgreSQL extension)	Hypertables with automatic time-based partitioning. Fast range queries, downsampling, and continuous aggregates.
Apache Flink or Kafka Streams	Real-time stream processing for windowed pattern detection over the event bus.
Prophet (Facebook/Meta)	Time-series forecasting for predicting future values of tracked metrics. Excellent for seasonality-aware baselines.
statsmodels (Python)	Statistical anomaly detection — z-score, CUSUM, exponential smoothing for simple cases.
Redis sorted sets	Efficient sliding window of recent events — add with timestamp score, query range by time.

Chapter 7 — Phase 6: Build the World Model

The world model is the crown of the architecture. Everything you have built so far — perception, memory, graph, temporal awareness — feeds into it. It is the single authoritative, continuously updated, causally aware representation of the current state of your environment.

What the World Model Is Not

The world model is not a database query. It is not a dashboard. It is not a static snapshot. It is a living object that is continuously reading from all memory layers and the live event stream, synthesizing their inputs into a coherent model, and making that model available for querying.

What the World Model Contains

Component	Description
Entity registry	The current state of every known entity — services, databases, users, agents, resources. Properties reflect the most recently observed values.
Relationship map	The current knowledge graph state. Which entities are connected, what their relationships are, and how strong each relationship is.
Temporal state	For each tracked metric, the current value, rate of change, trend direction, and deviation from baseline.
Active event context	The set of significant events from the last N minutes that are still 'open' — i.e., their consequences are still unfolding.
Causal chains	Known or inferred causal paths between events. 'Event A caused event B which is likely causing event C.'
Predictive state	Projected state of key entities and metrics 5, 15, and 60 minutes from now, based on current trends.
Anomaly registry	Currently active anomalies — deviations from baseline that have not yet been resolved.
Uncertainty map	For each assertion in the model, a confidence score. The model knows what it does not know.

How the World Model Gets Queried

The reasoning engine does not directly query Neo4j or PostgreSQL or Redis. It queries the world model through a single, unified API. The world model abstracts over all the underlying storage systems and provides a coherent view.

Example queries the reasoning engine might make:

- `get_current_situation()` — returns a compressed JSON summary of the most significant current anomalies, their affected entities, and their causal chains. This is the primary input to the reasoning engine.
- `get_entity_state(entity_id)` — returns the full current state of a specific entity, including its relationships, recent events, and temporal trends.
- `get_blast_radius(entity_id)` — traverses the knowledge graph to find all entities affected if this entity fails.
- `get_causal_chain(event_id)` — traces backward and forward through the graph to find what caused this event and what it is likely to cause.
- `get_prediction(metric_id, horizon_minutes)` — returns the predicted value of a metric at a future time point.

The Situation Assessor

Between the world model and the reasoning engine sits the situation assessor — a lightweight, fast component that compresses the full world model into a situation summary suitable for LLM reasoning.

The world model may contain thousands of entities and millions of events. An LLM context window is limited. The situation assessor acts as an intelligent filter: it ranks the current anomalies by severity and impact, selects the top N most significant, traces their causal chains, and formats them into a concise structured summary. This summary is what the reasoning engine actually receives.

Without the situation assessor, the reasoning engine is overwhelmed with data. With it, the reasoning engine receives a carefully curated, maximally informative brief — the equivalent of what an operations manager would say to a senior engineer who just walked into a war room: 'Here is what is wrong, here is what we know caused it, and here is what is at risk.'

Tech Stack for the World Model

Component	Technology
World model core	Python class with Redis-backed state. Subscribes to Kafka event bus and updates continuously.
Graph queries	Neo4j Python driver (neo4j package). Cypher queries for relationship traversal.
Time-series queries	SQLAlchemy + TimescaleDB for metric trend queries.
Semantic queries	Qdrant client for similar event retrieval.
Situation assessor	Custom Python logic — rule-based ranking and filtering, no LLM needed here.
World model API	FastAPI with async endpoints. Other components query via HTTP or direct Python import.

Chapter 8 — Phase 7: Build the Reasoning Engine

The reasoning engine is where the LLM lives. Not as the architecture — as one carefully bounded component within the architecture. Understanding this distinction is essential. The LLM does not perceive. The LLM does not remember. The LLM does not maintain the world model. The LLM reasons over a prepared, structured context and returns a decision.

What the Reasoning Engine Does

- Receives a situation summary from the world model (via the situation assessor)
- Optionally queries the world model for additional context about specific entities or relationships
- Retrieves relevant past experiences from episodic memory (similar situations and how they were handled)
- Retrieves relevant playbooks from procedural memory
- Constructs a prompt that combines all of this context
- Calls the LLM to produce a reasoned analysis and decision
- Parses the LLM response into a structured decision object
- Passes the decision to the planning system

Why the LLM Is Not the Architecture

This is the most important concept in this entire guide. Consider what happens if the LLM is the architecture:

- No persistent memory — every call starts from zero. The system forgets what happened last hour.
- No causal reasoning over structured data — the LLM cannot traverse a graph. It can only reason over text.
- No real-time grounding — the LLM's knowledge is frozen at training time. It does not know what is happening in your environment right now.
- High latency — an LLM call takes seconds. A perception event must be processed in milliseconds.
- High cost — calling an LLM for every event is economically unviable at scale.
- Hallucination risk — the LLM may confabulate facts about your environment that are simply wrong.

By contrast, when the LLM is one component within the architecture: the world model handles real-time grounding; memory provides continuity; the knowledge graph handles causal

reasoning; the situation assessor filters noise; and the LLM only reasons when a genuine reasoning step is needed — which may be once per minute rather than once per millisecond.

What Goes in the Reasoning Prompt

The prompt to the LLM is carefully structured. It contains:

- 8. System context — what role is this reasoning engine playing? What is its goal? What constraints apply? (e.g., 'You are a reasoning component in a cognitive architecture managing a production web application. Your goal is to analyze the current situation and recommend the safest and most effective remediation action.')
- 9. Situation summary — the compressed output from the situation assessor. The current anomalies, their severity, their affected entities, and their causal chains.
- 10. Relevant past experiences — 2–3 examples from episodic memory of similar situations and how they were resolved. Gives the LLM concrete precedent to reason from.
- 11. Available actions — the vocabulary of actions the system can take. The LLM must reason about which action is appropriate; it should not invent actions that don't exist.
- 12. Constraints — what the system is not allowed to do without human approval. What actions require confirmation.
- 13. Output format — a structured JSON schema that the LLM must produce. Not free text. The planning system needs a machine-parseable decision.

Decision Object Structure

The LLM must always return a structured decision. Define this schema strictly:

Field	Description
situation_assessment	Plain language summary of what the reasoning engine understands about the current situation.
root_cause_hypothesis	The most likely causal explanation for the observed symptoms, with a confidence score.
recommended_action	The specific action to take, selected from the available action vocabulary.
action_parameters	Parameters for the recommended action (e.g., which entity, what value, what target state).
confidence	How confident the reasoning engine is in this recommendation (0.0–1.0).
requires_human_approval	Boolean. True if this action is high-risk or irreversible.
alternative_actions	Array of 1–2 alternative actions if the recommended action is not available or fails.

reasoning_trace	Step-by-step explanation of how the conclusion was reached. Used for observability.
------------------------	---

When NOT to Use the LLM

The reasoning engine should have a fast path — a rule-based decision system that handles well-known situations without calling the LLM at all. If the world model shows a pattern that exactly matches a known playbook (e.g., queue depth above 10,000 for 5 minutes → scale worker pool), the planning system can execute directly without LLM reasoning. The LLM is reserved for novel situations, complex tradeoffs, and decisions that require multi-step causal reasoning.

Tech Stack for Reasoning

Component	Technology
LLM	Start with GPT-4o or Claude Sonnet via API. Later migrate to a local model (Mistral, LLaMA 3) for cost reduction.
LLM orchestration	LangGraph for stateful reasoning chains. LangChain for simpler single-step calls.
Output parsing	Pydantic for parsing and validating the structured JSON output from the LLM.
Prompt management	Store prompt templates in version-controlled files. Never hardcode prompts.
Rule engine (fast path)	Simple Python conditions or a lightweight rule engine (business-rules library).

Chapter 9 — Phase 8: Build the Planning System

The planning system converts a reasoning decision into a concrete, executable strategy. It is the bridge between 'what should be done' (reasoning) and 'what is actually done' (execution). A good planning system does not just translate decisions into steps — it structures those steps with dependencies, contingencies, and rollback procedures.

Why Planning Is a Separate Layer

The reasoning engine produces a high-level decision: 'scale the read replicas.' The execution layer needs concrete instructions: 'call this API endpoint, with these parameters, wait for this confirmation signal, then verify this metric crosses this threshold within this timeout, and if not, execute this rollback procedure.' The planning layer bridges that gap.

The Plan Structure

Component	Description
Goal	The high-level objective this plan achieves. Must be measurable — the planning system needs to know when the goal is met.
Preconditions	Conditions that must be true before plan execution begins. If preconditions are not met, the plan waits or aborts.
Steps	Ordered list of concrete actions. Each step has: action type, parameters, expected outcome, timeout, and on_failure behavior.
Dependencies	Which steps must complete before others can begin. Supports parallel execution where steps are independent.
Success criteria	Observable conditions that indicate the goal has been achieved. The feedback loop checks these.
Rollback plan	The sequence of actions to take if the plan fails midway. Every irreversible action needs a compensating action.
Approval gates	Steps that require human confirmation before proceeding. The plan pauses and waits for explicit approval.

Planning Algorithms

For most practical cognitive architectures, you do not need A* or Monte Carlo Tree Search initially. Start with:

- Template-based planning — retrieve the most similar past plan from procedural memory, parameterize it for the current situation. Works well for known problem types.
- LLM-assisted plan generation — pass the decision object to the LLM with a prompt asking it to generate a plan using the available action vocabulary. Validate the output against the plan schema.
- Hierarchical decomposition — for complex goals, break them into sub-goals, generate a plan for each sub-goal, then compose them. This is the basis of hierarchical planning.

Monte Carlo Tree Search and reinforcement learning-based planning become relevant once you have enough episodic memory to train on (typically after 6–12 months of operation). Add them later.

Plan Execution Monitoring

The planning system does not just generate plans and hand them to execution. It monitors each step's completion, checks success criteria, and handles failures. Every step completion is an event that flows back through the perception layer, updating the world model, so the reasoning engine always has current information about plan progress.

Chapter 10 — Phase 9: Build the Execution Layer

The execution layer is where plans become reality. It is the system’s hands — the components that actually make changes in the environment. Everything upstream has been about understanding and deciding. The execution layer acts.

The Execution Principle

Every action in the execution layer must satisfy three requirements: it must be observable (the perception layer can detect its effects), it must be traceable (the action is recorded in episodic memory with full context), and where possible it must be reversible (a rollback action exists in the plan).

Action Types

Action Type	Examples	Key Requirement
API call	REST/GraphQL calls to external services, webhook triggers, cloud provider APIs	Idempotent design; retry logic with exponential backoff; timeout handling
File operation	Config file updates, log writes, artifact creation, code deployment	Atomic writes; backup original before modifying; verify write succeeded
Process management	Start/stop/restart services, spawn worker processes, kill hung processes	Health check after action; process supervision to detect unexpected exit
Sub-agent delegation	Spawn a specialized agent for a subtask; send task to agent pool; aggregate results	Clear task specification; timeout with escalation; result validation
Message emission	Alert notifications, Slack messages, emails, queue messages, event bus publications	Delivery confirmation; deduplication to prevent alert storms
State mutation	Direct database writes, cache invalidation, configuration changes	Transaction support; optimistic locking; change event published to perception layer
Human handoff	Pause execution, present situation to human operator, wait for approval or instruction	Clear context presentation; timeout with escalation path; record human decision

The Execution Feedback Contract

Every action must produce a feedback event. This is not optional. The feedback event has the same structure as any other perception event — it goes through the normalizer, onto the event bus, into memory, and into the world model. The feedback event includes: which action was taken, what the target entity was, what the expected outcome was, whether the action succeeded, and the observed outcome.

Without this contract, the feedback loop breaks. The system acts but cannot learn. The world model does not update. The reasoning engine cannot tell whether its recommendation worked. The entire adaptive quality of the architecture depends on action feedback flowing back through perception.

Chapter 11 — Phase 10: Build the Feedback Loop

The feedback loop is the most neglected component in most cognitive architecture projects. It is what separates a pipeline from an adaptive system. It is also the most conceptually important: without it, your system can act but it cannot learn from acting.

What the Feedback Loop Does

The feedback loop has three responsibilities:

14. Verification — after an action is executed, verify that the intended effect occurred. Did scaling the database replicas actually reduce latency? Did restarting the service actually clear the error queue? Verification events flow through the perception layer and update the world model.
15. Outcome recording — record the action, its context (what the world model looked like when the decision was made), and its outcome in episodic memory. This is the training data for everything that follows.
16. Model updating — update the knowledge graph and procedural memory based on the outcome. A successful action reinforces the strategy that produced it. A failed action weakens it. Over time the system builds an empirical understanding of what works.

The Feedback Cycle in Detail

Here is the precise sequence:

17. The execution layer completes an action step and publishes a `step_completed` event to the event bus.
18. The perception layer receives this event, normalizes it, and routes it to memory. The world model updates with the new state.
19. The planning system checks whether the action's expected outcome was observed (using the world model's current state). If yes, proceed to the next step. If no, trigger the `on_failure` handler.
20. After the full plan completes, the situation assessor checks whether the original goal's success criteria have been met.
21. The outcome (success, partial success, or failure) is written to episodic memory with full context: situation summary, decision made, plan executed, outcome observed.
22. The procedural memory updates the strategy's success rate. If this plan type has succeeded 8 times and failed 2, its confidence score is 0.8.
23. The knowledge graph updates causal relationships. If action X consistently resolves situation Y, add or strengthen the edge `RESOLVES` between them.

Each iteration of the feedback loop makes the system marginally better. After 100 cycles it is noticeably better. After 1,000 cycles it is dramatically better. This is how the cognitive architecture develops genuine expertise over time — not through retraining a model, but through the accumulation of structured, contextualized experience in its memory systems.

The Observability Layer

The observability layer is the feedback loop's external face — the set of tools that allow you, the developer, to understand what the system is doing and why. Every component must emit traces:

- Every reasoning call: what situation summary was provided, what decision was returned, how confident was the LLM, how long did it take.
- Every plan execution: which steps were executed in what order, which succeeded and which failed.
- Every world model state: what was the world model's assessment at key decision points.
- Every memory operation: what was stored, what was retrieved, what was the similarity score.

Tool	Purpose
OpenTelemetry	Distributed tracing across all components. Traces a single decision from perception event through reasoning through execution.
Prometheus + Grafana	Metrics dashboards — reasoning latency, event throughput, memory write rates, plan success rates.
LangSmith (if using LangChain/LangGraph)	LLM call tracing — inspect every prompt and response, monitor latency and cost.
Structured logging (structlog)	Every significant event in every component emits a structured log that can be queried and analyzed.

Chapter 12 — Phase 11: Multi-Agent Coordination

Once the single-agent loop — perceive, remember, model, reason, plan, execute, feedback — is proven and stable, you are ready to scale cognition across multiple agents. Multi-agent coordination multiplies the system's capability: tasks can be parallelized, agents can specialize, and the system can handle complexity that would overwhelm a single reasoning thread.

Why Multiple Agents?

A single agent processes one decision at a time. Real environments generate many simultaneous demands. A hospital cognitive architecture might simultaneously need to reason about patient deterioration in ward A, optimize staff scheduling for the night shift, and monitor supply inventory — three independent problem streams that can be handled in parallel.

Additionally, specialization improves performance. A monitoring agent that focuses only on anomaly detection becomes better at that specific task than a generalist agent that also handles scheduling and planning. Specialization allows each agent to have a more focused prompt, more relevant episodic memory, and more appropriate action vocabulary.

The Shared Perception Contract

All agents share the same event bus and the same world model. No agent has a private perception stream. This is the critical constraint that keeps multi-agent systems coherent. If agents had private perception, they would develop divergent world models and conflict in their actions. A shared bus ensures that every agent sees the same events and the world model reflects all agents' observations.

Agents write to the bus by publishing `agent_event` type events through the perception layer's normalization pipeline. They never write directly to memory or the knowledge graph. The world model is the single source of truth.

Agent Types

Agent Type	Role
Monitor agent	Continuously observes the event stream and world model. Detects anomalies and triggers the reasoning engine when significant situations arise. Always running.
Planner agent	Receives situations from the monitor agent, reasons about them, and generates plans. May call specialist agents for domain-specific reasoning.
Executor agent	Carries out plan steps. May be specialized by action type — one agent handles API calls, another handles database operations, another handles human communication.
Memory agent	Manages memory consolidation — moves events from working memory to episodic, updates graph relationships, consolidates and summarizes old episodic memories.
Prediction agent	Runs predictive models over the temporal state and publishes predicted_state events to the bus. The world model incorporates these predictions.
Specialist agent	Domain expert for a specific area — a security specialist, a database performance specialist, a scheduling optimizer. Spawned on demand by the planner.

Agent Communication

Agents communicate through two channels:

- The event bus — for asynchronous, broadcast communication. An agent publishes an event; any other agent subscribed to that event type receives it.
- Direct task delegation — for synchronous, targeted communication. A planner agent creates a Task node in the knowledge graph, assigns it to a specific executor agent, and waits for the task's completion event on the bus.

CONFLICT RESOLUTION

When two agents reach conflicting decisions about the same entity, the planner agent acts as arbiter. Conflicts are detected by the knowledge graph (two agents both have active tasks targeting the same entity). The planner compares their plans, assesses risk, and either merges them or prioritizes one. This must be designed explicitly — do not assume conflicts will not occur.

Chapter 13 — Phase 12: Predictive Dynamic Models

This is the most advanced phase. Do not attempt it until the first eleven phases are stable and you have at least three to six months of episodic memory accumulated. Predictive models require data to train on — data that your earlier phases have been collecting.

What Predictive Models Add

Everything built so far allows the system to react intelligently to events. Predictive models allow it to act before events occur — to see a pattern developing and intervene while there is still time, rather than responding after the situation has already degraded.

Graph Neural Networks (GNNs)

Your knowledge graph is a perfect input structure for a GNN. A GNN learns how state propagates through a graph — if node A changes, how does that affect its neighbors, and their neighbors? This is directly applicable to failure propagation, resource bottleneck prediction, and cascading dependency effects.

Train a GNN on your historical episodic data: given the state of the graph at time T, predict the state at time T+15 minutes. After sufficient training, the GNN becomes a faster, cheaper alternative to LLM-based reasoning for routine predictions.

When to Use GNNs vs LLMs

Use GNN when	Use LLM when
The situation matches patterns seen many times before	The situation is novel with no close historical precedent
Speed is critical (millisecond predictions)	Complex multi-step causal reasoning is required
The decision is structured and bounded	The decision requires natural language understanding

Training data is available (1000+ similar past examples)	Insufficient training data exists for this situation type
---	---

Tech Stack for Predictive Models

Tool	Purpose
PyTorch Geometric	GNN framework. Natively works with graph-structured data from Neo4j.
DGL (Deep Graph Library)	Alternative GNN framework, well-documented for knowledge graph tasks.
PyTorch Forecasting	Time-series forecasting models (Temporal Fusion Transformer) that outperform Prophet for complex multivariate forecasting.
MLflow	Experiment tracking and model registry. Essential once you have multiple models to manage.
Triton Inference Server	High-performance model serving when prediction latency becomes a bottleneck.

Chapter 14 — Your First Prototype

Theory without execution is just planning. Here is exactly what to build as your first working prototype — scoped to be completable in 4–8 weeks while demonstrating every core concept.

The Prototype Goal

Build a cognitive monitor for any environment you have access to — your own computer, a small web application, a Raspberry Pi cluster, a simulated game world. The prototype must:

- Continuously perceive at least three distinct signal sources
- Store events in episodic memory and build a simple knowledge graph
- Detect at least one temporal pattern
- Use an LLM to reason about detected anomalies
- Produce a structured recommendation (do not auto-execute yet)
- Show that the feedback loop updates the world model after each reasoning cycle

Concrete Build Order

24. Week 1: Define the event schema for your chosen environment. Build two normalizers. Get events flowing into a Kafka topic. Verify schema validation works.
25. Week 2: Set up Redis (working memory) and PostgreSQL with TimescaleDB (episodic memory). Build the memory router. Verify events are being stored correctly in both.
26. Week 3: Set up Neo4j. Build the graph updater that reads entity_refs from events and creates/updates nodes and edges. Visualize the graph in the Neo4j browser — you should start seeing your environment take shape.
27. Week 4: Build the temporal state tracker. Track at least one metric over time and implement a simple anomaly detector (z-score against rolling baseline). Verify it fires correctly.
28. Week 5: Build the world model API — a Python class that combines graph queries, temporal state, and recent events into a coherent situation summary.
29. Week 6: Build the situation assessor and reasoning engine. Wire up an LLM call. Test it with a synthetic anomaly — does the reasoning make sense?
30. Week 7: Build the observability layer. Add structured logging and a simple Grafana dashboard. Now you can see everything the system is doing.
31. Week 8: Run the system for a week. Collect feedback. Identify the three biggest gaps. Fix them.

RESIST THE URGE TO AUTO-EXECUTE

In your first prototype, make all actions recommendations only — display them to a human, log them, but do not auto-execute. This lets you validate that the reasoning is sound before giving the system the ability to actually change things. You will catch many reasoning errors in this phase that would have caused real problems if auto-executed.

The Complete Tech Stack for the Prototype

Layer	Technology	Why
Event schema	Pydantic v2	Fast validation, auto-documentation, type safety
Event bus	Redpanda (Docker)	Kafka-compatible, simpler setup, excellent for local dev
Working memory	Redis 7 (Docker)	Sub-millisecond, TTL support, pub/sub built in
Episodic memory	PostgreSQL + TimescaleDB (Docker)	Time-series queries, full SQL, free
Vector search	Qdrant (Docker)	Open source, runs locally, excellent Python client
Knowledge graph	Neo4j Community (Docker)	Free, full Cypher, browser visualization
API framework	FastAPI + Uvicorn	Async, fast, auto-generates OpenAPI docs
LLM	Claude Sonnet or GPT-4o via API	Start with API, migrate local later
LLM orchestration	LangGraph	Stateful agent graphs, good observability

Task queue	Celery + Redis	Background tasks for perception polling
Observability	Structlog + Prometheus + Grafana	Structured logs + metrics dashboard
Container orchestration	Docker Compose	Run entire stack locally with one command

Chapter 15 — Long-Term Evolution

The cognitive architecture you are building is not a finished product — it is a growing substrate. Here is how it evolves over time:

Stage 1 — Months 1–3: Single-Environment Cognition

Build the prototype. Prove the loop works. Accumulate episodic data. Focus on the quality of the event schema and the accuracy of the world model. Success criterion: the system correctly identifies 80%+ of significant situations in your environment and produces reasonable recommendations.

Stage 2 — Months 3–6: Adaptive Strategies

Enable auto-execution for low-risk actions. Build the feedback loop completely. Begin accumulating outcome data. The procedural memory should start showing learned strategies. Success criterion: the system successfully auto-resolves at least 30% of known problem types without human intervention.

Stage 3 — Months 6–12: Multi-Agent Scale

Introduce specialized agents. Build the agent coordination protocol. Expand the knowledge graph to cover more entity types and relationships. Begin training simple predictive models on your accumulated episodic data. Success criterion: the system handles parallel problem streams without human coordination.

Stage 4 — Year 1–2: Predictive Intelligence

Deploy trained GNNs for known failure modes. Implement proactive intervention — the system acts before problems manifest rather than after. Expand to new environments. Success criterion: the system prevents more problems than it reacts to.

Stage 5 — Year 2+: General Environmental Intelligence

Apply the architecture to multiple, qualitatively different environments. Explore transfer learning between environments. Consider physical embodiment — the same architecture that manages a software environment can, with appropriate perception adapters and execution vocabulary, manage a physical one.

FI
N
A
L
P
RI
N
CI
P
L
E

Every decision in this architecture should be made in service of one goal: maintaining the most accurate, most causally rich, most temporally aware model of your environment possible. The reasoning, planning, and execution are only as good as the model they operate on. Invest disproportionately in perception quality, memory design, and world model accuracy. That is where the intelligence lives.

The architecture is the intelligence.

Build the substrate. The capabilities follow.